

Frontend Hosting Architecture using AWS

Powered by VPC, EC2, Route 53, ACM, ELB, ASG, and CloudFront

Presented by Antolawrence

 by Antolawrence



Overview of the Architecture

VPC

Private, controlled network environment

EC2 + ELB

App hosted with load balancing

ACM

Provides SSL for secure HTTPS access

Auto Scaling

Adjusts capacity based on traffic

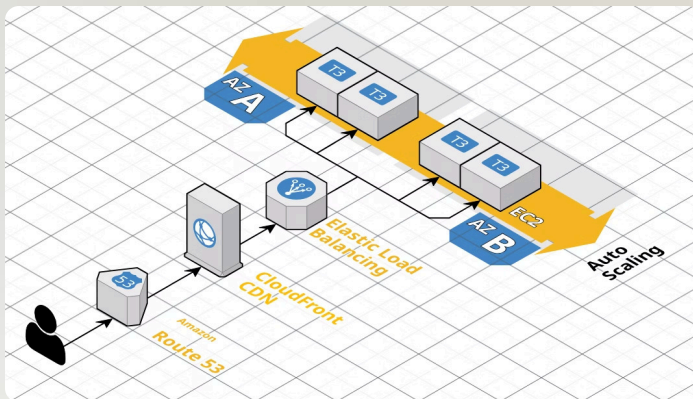
Route 53

Domain routing and DNS management

CloudFront

Global content delivery network





Architecture Diagram

Visual overview integrating all core AWS components

VPC and Networking



Custom VPC

Isolated and secure virtual network



Public Subnet

Hosts load balancers and gateways



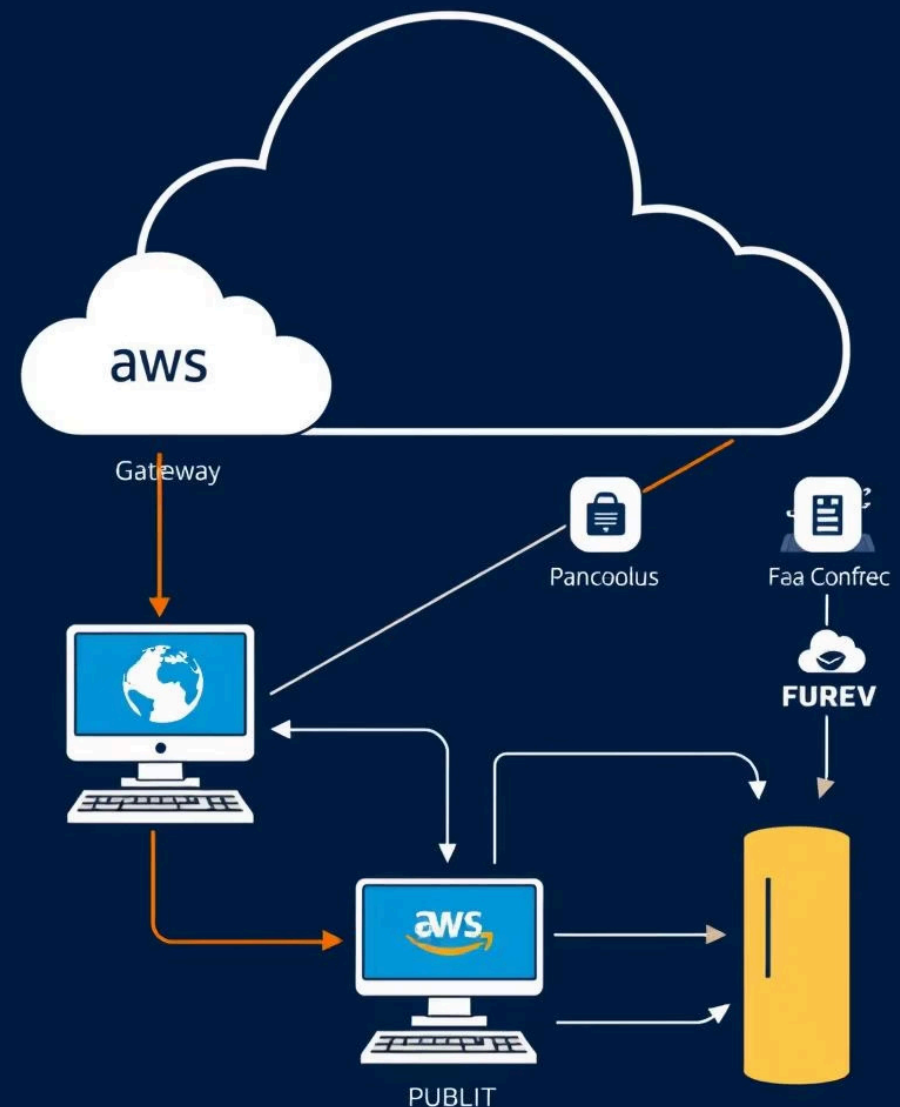
Private Subnet

Hosts EC2 instances securely



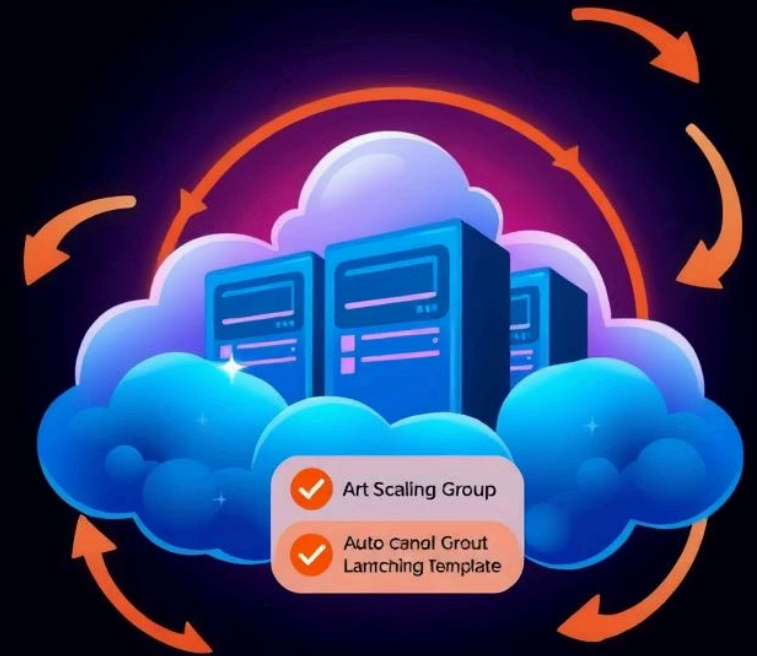
Internet Gateway

Provides controlled internet access



EC2 and Auto Scaling Group

- **EC2 Hosts**
Deploy frontend application code
- **Auto Scaling Group**
Ensures availability and scalability
- **Custom AMI**
Standardizes instance configuration
- **Launch Template & Health Checks**
Automates instance management and monitoring



Elastic Load Balancer

Load Balancer Role

Efficiently routes incoming traffic

ACM Integration

Provides secure HTTPS connections

Target Group

Directs traffic to healthy EC2 instances



CloudFront (CDN)

CloudFront CDN

Caches static content
worldwide

Positioned Before ALB

Serves cached content
instantly

Speeds Up Delivery

Reduces latency for global
users

Benefits of the Architecture

High Availability
Redundant resources and auto healing

Decoupled Frontend
Scalable and modular architecture



Auto Scaling
Handles variable traffic loads

Secure HTTPS
Encryption with ACM certificates

Global Content Delivery
CloudFront caches globally

Conclusion & Key Takeaways

Scalability

Auto Scaling adapts capacity in real time

Security

ACM SSL and VPC isolation protect data

High Availability

Load Balancer and Route 53 ensure uptime

Global Performance

CloudFront reduces latency worldwide

Cost Efficiency

Dynamic resource management reduces spend